How to Discuss Caching in System Design

systemdesignschool.io/fundamentals/cache-interview-checklist

Interview Walkthrough

Candidates often mention caching as a one-liner: "We'll add Redis here for performance." This fails to demonstrate understanding. Interviewers want to see reasoning—why cache this data, which pattern, what happens on failure. This article walks through eight dimensions with a running example to show how to discuss caching with depth.

The Running Example

Throughout this walkthrough, we'll use an **e-commerce product page** as the scenario. The page shows product details, pricing, reviews summary, and seller information. The system handles 50,000 page views per second at peak, with product data updating a few times per day.

1. What to Cache

Start by identifying which data is worth caching. Not everything benefits equally from a cache—the best candidates are read frequently and change infrequently, though caches also help with expensive computations and session state.

The thought process. Look at the read-to-write ratio. Product details are read 50,000 times per second but updated perhaps a few times per day—an astronomically skewed read-to-write ratio and an ideal caching candidate. Compare this to a user's shopping cart, which mutates on every click. A cache-aside approach for the cart has limited benefit because the cached entry invalidates on every mutation (though Redis is often used as the cart's primary store).

What to say. "The product detail query runs on every page view—50,000 QPS at peak. The data changes a few times per day. This extreme read-to-write ratio makes it a strong caching candidate. The shopping cart, by contrast, changes on every interaction, so caching it provides minimal benefit."

Common mistake. Saying "cache everything" without reasoning. The interviewer wants to see you distinguish hot data from cold data. Another mistake: caching only at one layer. Consider whether both the database query result (app cache) and the rendered response (CDN) deserve caching.

What the interviewer is looking for. Can you identify the specific queries or data paths that create database load? Can you quantify the benefit (hit rate, load reduction)?

2. Cache Key Design

After deciding what to cache, explain how keys are structured. This shows attention to correctness and future-proofing.

The thought process. A product page cache key needs to uniquely identify the content. The key `product:v1:detail:12345` encodes: the entity type (product), a schema version (v1), the data scope (detail), and the identifier (12345). If the product detail schema changes (a new field is added), bumping to `v2` avoids serving old-format data from cache.

What to say. "I'd use keys like `product:v1:detail:{id}`. The version segment lets us deploy schema changes safely—new code writes to v2 keys, old v1 entries expire naturally via TTL. For the reviews summary, a separate key: `product:v1:reviews:{id}` with a shorter TTL since reviews change more frequently."

Common mistake. Using overly generic keys (`product:12345`) that collide when different views of the same product are cached (detail view vs list view vs admin view). Another mistake: forgetting versioning, which forces cache flushes on every deploy.

What the interviewer is looking for. Do you think about key collisions, versioning, and how different data types have different freshness requirements?

3. Read and Write Patterns

State which caching pattern you're using and why. Default to cache-aside unless there's a specific reason for another pattern.

The thought process. Product pages are read-heavy with infrequent writes. Cache-aside (read: check cache → miss → query DB → store in cache; write: update DB → delete cache key) is the natural fit. The application controls all cache interactions, and the cache layer stays simple (just Redis as a key-value store).

If this were an inventory count updated on every purchase (write-heavy), write-behind might apply—buffer frequent updates in cache and flush to the database periodically. But the durability risk (lost writes on crash) makes write-behind inappropriate for inventory where accuracy matters.

What to say. "For product details, I'd use cache-aside. Reads check Redis first, fall back to PostgreSQL on miss, and store the result. On product update, we delete the cache key—the next read repopulates with fresh data. Write-through would add unnecessary latency since product reads far exceed writes, and most writes won't be read again immediately."

Common mistake. Choosing write-through "for consistency" without considering that write-around (delete on write) achieves the same consistency with less cache pollution. Another mistake: not addressing the write path at all—only discussing reads.

What the interviewer is looking for. Can you reason about the tradeoff between consistency and performance? Do you understand when each pattern applies?

4. Invalidation Strategy

Explain how stale data is prevented. This is where candidates often lack depth.

The thought process. Two mechanisms work together. Explicit invalidation (delete the key when the product is updated) handles the normal case—on the next read, the cache repopulates with fresh data. TTL acts as a safety net—if the invalidation message is lost or the delete fails, the stale entry self-destructs after a bounded time.

For product details that change a few times per day, a 15-minute TTL means the worst-case staleness is 15 minutes (if invalidation fails). For pricing that must be accurate, a 60-second TTL tightens the window. For reviews summaries (where slight staleness is invisible to users), 5–10 minutes is acceptable.

What to say. "Each data type gets a TTL matching its staleness tolerance: product details at 15 minutes, pricing at 60 seconds, reviews at 5 minutes. On any product write, we explicitly delete the cache key—so the normal-case staleness is near zero. The TTL is a safety net for when invalidation fails. I'd also add jitter (±10%) to prevent batch-loaded products from expiring simultaneously."

Common mistake. Setting one TTL for all data types. Another: relying only on TTL without explicit invalidation (accepting up to 15 minutes of staleness on every write). The worst mistake: no invalidation strategy at all—"the cache will eventually expire."

What the interviewer is looking for. Do you understand that different data types need different freshness guarantees? Can you layer explicit + TTL-based invalidation?

5. Eviction Policy

When the cache fills, something must be removed. State the policy and why.

The thought process. LRU (least recently used) is the default for most web workloads because recent access predicts future access—users browse a category and revisit the same products within minutes. If the system runs nightly batch jobs that scan the entire product catalog (for indexing or analytics), LFU (least frequently used) resists the scan pollution that would push hot products out of cache.

What to say. "LRU as the default. If we run catalog scans that pollute the cache, I'd switch to LFU—it protects frequently-accessed products from being evicted by one-time batch reads. For sizing, the hot 20% of products (roughly 200K entries at 2KB each) needs about 400MB—well within a single Redis node."

Common mistake. Ignoring eviction entirely. Another: over-engineering by proposing custom eviction policies when LRU handles most cases.

What the interviewer is looking for. Do you understand the memory tradeoff? Can you size the cache with rough math?

6. Scaling

When one node isn't enough, explain how to distribute the cache.

The thought process. A single Redis node handles ~100K–200K operations per second. Our 50,000 product page views generate 50K cache reads per second—one node suffices for now. But if traffic doubles or the working set exceeds one node's memory, sharding across multiple nodes becomes necessary.

Redis Cluster splits the key space into 16,384 hash slots distributed across nodes. Adding a node only remaps keys in the transferred slots (~25% for 3→4 nodes). For read scaling, each shard can have replicas that serve read traffic.

What to say. "At 50K QPS with typical payload sizes, a single Redis node likely handles the load. If we grow beyond a single node's throughput or memory capacity, I'd shard using Redis Cluster. Hash slots give us incremental scaling—adding a node only remaps ~25% of keys. For the viral-product hot-key problem, I'd use local in-process caching with a 5-second TTL to absorb repeated reads before they hit Redis."

Common mistake. Proposing sharding for a system that doesn't need it (over-engineering). Another: ignoring the hot key problem—sharding distributes keys evenly by hash, not by access frequency.

What the interviewer is looking for. Can you identify when scaling is actually needed? Do you understand the hot key problem that sharding doesn't solve?

7. Failure Modes

Name the relevant failure mode for the scenario and its defense. This shows senior-level awareness.

The thought process. For a product page system, the primary risk is **stampede** (thundering herd). A popular product's cache entry expires, and thousands of concurrent requests all miss simultaneously and flood the database with identical queries.

Cache penetration is relevant if the system allows user-generated product searches—users can query non-existent product IDs. Avalanche applies if products are batch-loaded with identical TTLs.

What to say. "The main risk is stampede on popular products. When a viral item's cache expires, 5,000 concurrent requests all miss and hit the database. I'd use request coalescing—the first miss acquires a lock, fetches from DB, and populates the cache. The other 4,999 requests wait for the lock and read from cache. For user-facing search by ID, I'd add a Bloom filter to reject non-existent products before they reach the database."

Common mistake. Not knowing any failure modes—just saying "the cache could go down." Another: naming all three modes without identifying which is most relevant to the specific scenario.

What the interviewer is looking for. Can you identify the specific risk for this system? Can you name a concrete defense and explain the mechanism?

8. Consistency Tradeoff

Acknowledge that caching trades consistency for performance. Frame the acceptable staleness per data type.

The thought process. Different fields on the same product page have different staleness tolerances. The product name and images can be stale for hours (they rarely change). The price must be current (a stale price leads to customer complaints or financial loss). The review count can be stale for minutes (users don't notice if it's 4,201 vs 4,203).

What to say. "Caching trades consistency for performance, and the acceptable tradeoff varies by data type. Pricing gets a 60-second TTL with synchronous invalidation on update—the worst case is 60 seconds of stale pricing. Product descriptions use 15-minute TTL—staleness here is invisible to users. Inventory count bypasses the cache entirely and queries the database directly, because overselling is worse than a few milliseconds of added latency."

Common mistake. Treating all data the same ("everything gets a 5-minute TTL"). Another: claiming the cache provides "strong consistency" when it fundamentally does not. The worst mistake: never mentioning the consistency tradeoff at all.

What the interviewer is looking for. Do you understand that caching is a tradeoff, not free performance? Can you articulate what level of staleness is acceptable for different data types?

Matching Depth to the Problem

Not every interview requires all eight points. Focus on what matters for the specific system being designed.

System Type	Emphasize	De-emphasize
Social media feed	Read patterns, stampede, hot keys	Write patterns, security
Payment system	Consistency, what not to cache, security	Eviction, scaling
E-commerce catalog	All eight dimensions apply	—
Real-time dashboard	Write-behind, observability, staleness	Eviction, penetration
Chat application	Session caching, HA, scaling	Write patterns, avalanche

The goal is demonstrating reasoning, not reciting a list. Pick the 3–4 dimensions most relevant to the problem and discuss them with depth rather than covering all eight superficially.